

PROJECT OVERVIEW

REVISION 0, 06/21/26

INTRODUCTION

The intent of this document is to provide a high level summary of the programs that are used to demonstrate how the JSON Remote Procedure Call (RPC) protocol can be used in conjunction with Named Pipes on a Windows PC to allow for bidirectional communication of data and text between a Visual Basic .NET (VB.NET) client program and a Fortran server program.

BACKGROUND

Fortran

The original version of Fortran was released in 1957. At that time, there were no displays, nor were there any keyboards, mice, trackballs, etc. to allow for the user to interact with any program. As such, Fortran programs were typed out onto punch cards (the source code), and the results were printed out on paper (the output). The user interface concepts of a display, keyboard, mice, disks, cloud storage, windows, buttons, etc. would not be prevalent until decades later with the introduction of the Macintosh and Windows 1.0 (as well as other systems).

Even though Fortran is still alive and kicking, with the most current version of the program noted as 2023, Fortran remains somewhat of a “text only” language. There are a variety of Fortran compilers that run on Windows, some free and some paid. For the purposes of this project, the current version of the Intel Fortran IFX compiler was used in the Microsoft Visual Studio (VS) development environment.

There are basically 3 ways the Fortran program can have an interface Windows:

- Console only
- QuickWin window
- Full Windows program

The console interface is the most common, and is used in this project. The text is displayed in a window, and scrolls up as more text is added; simple, but it works. The QuickWin option allows for the generation of a true Windows window, and the text generated by the Fortran program is displayed within the window; better, functional, but not that much of an improvement over the console option (can't add buttons or other objects). A fully compliant Windows based program can be generated in Fortran, but the programmer must implement everything from scratch using Windows Application Programming Interfaces, or APIs. The Intel Fortran compiler package and documentation does offer a ‘starter’ version, but is somewhat limited and can be somewhat tedious to implement a fully functional program.

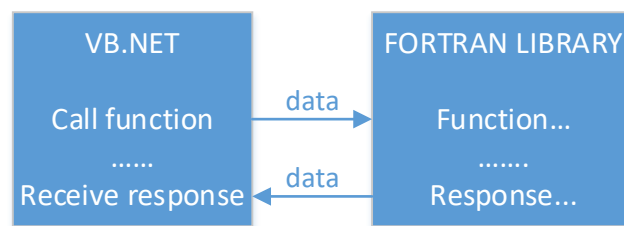
Visual Basic

In 2026, there are several operating systems, with user interfaces that provide for the input and display of information on displays, and allow the user to directly type in data, manipulate windows, click on objects such as buttons, sliders, etc., save output to digital storage (jump drive, cloud, etc.) and send to a printer (paper copy, PDF, etc.). Now there are a plethora of programming languages that allow for the (relatively) easy creation of Windows based programs [note – for the purposes of this project, the focus is on Microsoft Windows environment, and not MacOS, Linux, or other operating systems and interfaces]. Microsoft offers their development platform, Visual Studio, which allows for the use of several programming languages that collectively are under the “.NET” umbrella. The most popular of these (currently) is C#.NET. However, for the purposes of this project, Visual Basic, or VB.NET, was chosen to develop the Windows programs, due to the familiarity of the author with VB.NET in lieu of C#.

VB.NET allows for the easy and rapid creation and modification of a Windows program, consisting of objects such as forms, button, text display and text input boxes, etc. Adding other functions such as saving programs to disk, cloud, etc. printing output, and other functionalities can be generated in Visual Studio with relative ease.

Fortran and Visual Basic

So if you want your Fortran program to run in Windows, but do not want to use any of its three ‘native’ interfaces (listed above), one can use Visual Basic (as well as many other languages) as the ‘front end’ Windows interface, and Fortran for the number crunching. This is typically implemented as a standalone VB.NET program that sends and receives data from a Fortran static library, where the static library consists of one or more functions or subroutines.



For many applications this is more than adequate. There are, however, some limitations. These include:

- **Numbers:** Fortran typically exchanges data with non-Fortran programs through **language bindings** such as C interoperability features defined in the ISO_C_BINDING module, which map Fortran types to equivalent C types. Variables are transferred by passing pointers, references, or raw memory buffers using **interop conventions** that both languages agree on. More complex data is often handled through **wrapper functions** that translate structures and calling conventions between languages. This is true for VB.NET and other .NET languages. Different languages—including .NET languages and Fortran—use different **data models**, memory layouts, and **calling conventions**, so they can’t directly pass variables without an agreed-upon interface.

However, the incompatibility isn't because Fortran is uniquely odd; it's because *every* language ecosystem defines its own rules, so interoperability requires **marshalling** or standardized bindings like ISO_C_BINDING to translate data safely between them.

- Text: Transferring text between Fortran and .NET is absolutely possible, but it's the *messiest* part of interoperability because Fortran's **fixed-length character model** is fundamentally different from .NET's **managed Unicode strings**. The two sides must agree on a representation—usually **C-style null-terminated strings**, **explicit length arguments**, or **marshalled byte buffers**—and both languages must handle encoding and memory ownership carefully. In practice, most developers use small **interop wrappers** in C to translate between Fortran CHARACTER variables and .NET's System.String, because C is the common denominator both sides understand.
- Matrices: For **numeric matrices**, interoperability is straightforward: Fortran stores arrays in **column-major order**, while .NET (via VB.NET) stores them in **row-major order**, so both sides must agree on layout and pass a pointer to a contiguous block of memory. Once the layout is agreed upon, you can exchange matrices by passing raw buffers (e.g., double*) using **C-interop conventions**, and each side indexes the data according to its own rules.
- For **text matrices**, things get much harder: Fortran uses **fixed-length CHARACTER arrays**, while .NET uses **variable-length Unicode strings**, so every element must be marshalled individually—either as null-terminated C strings, explicit-length strings, or fixed-width byte buffers. Because of this mismatch, developers almost always introduce a **C wrapper layer** that converts each Fortran CHARACTER element into a C string before .NET receives it.
- Communication, status, control, etc.: The Fortran functions in a static library are basically a 'black box' that receives input, and provides a response. This is perfectly acceptable for most applications. But many Fortran applications are heavy duty number crunchers, that can take a considerable amount of time to run, even on a modern PC. As such, there is no native method or mechanism for the Fortran code to send a status, error message or other information (other than the response) to the VB.NET program. Likewise, for a long running application, the user may wish to have the ability to 'pause/resume' execution of the Fortran code, or even 'exit' the execution.

Once again, it can be assumed that for previous applications that either these limitations were not valid limitations, or have already been previously resolved through creative coding.

Project basis

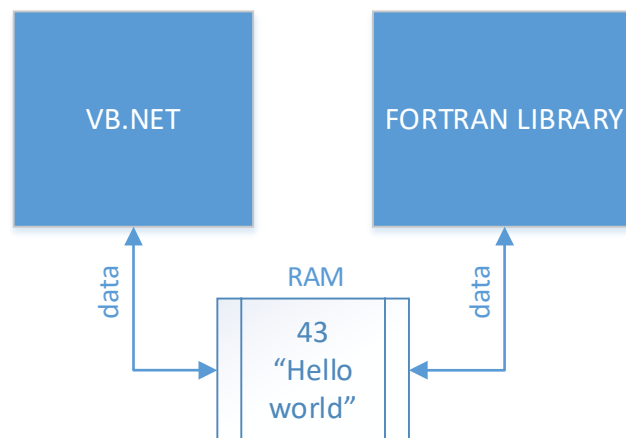
The objective of this project is to provide an alternate methodology for the bi-directional transfer of data (numerical values and text) between a VB.NET program and a Fortran program. Note that the Fortran portion is a separate *program*, and not a *static library*; more on that later. It is acknowledged that this methodology would be more applicable to 'large/complex' Fortran applications, due to the potential level of effort to implement.

Communication

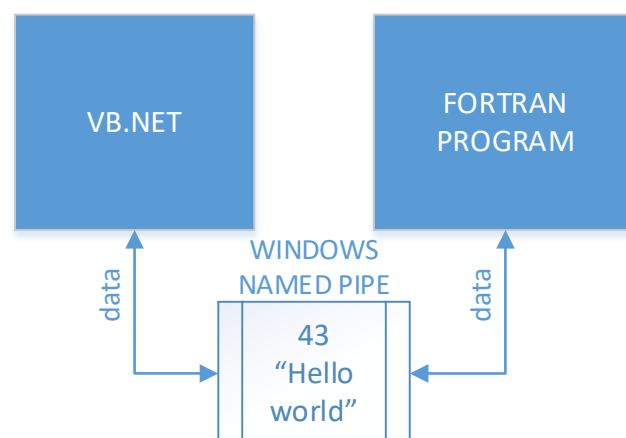
As noted in a previous section, Windows saves data (numbers, text, etc.) in memory on the PC. Due to several considerations, values such as "43.5" and "Hello world" stored at a physical address in RAM is unfortunately not the same for VB.NET as it is for Fortran. As an alternative, we can provide a different

method of transferring data between programs, as opposed to shared data between a VB program and a Fortran function. For this project, the use of Windows **Named Pipes** was chosen as the transfer mechanism. Windows Named Pipes provide a bidirectional communication channel that lets separate processes (programs) exchange data using a shared, stream-like interface. They support local or network communication and behave much like files, with one side writing and the other reading through a **named pipe endpoint**. Because they're managed by the Windows kernel, they offer reliable, secure inter-process communication (IPC) with features like **access control** and **message boundaries**.

So, instead of transferring data via shared memory,



We have two separate running processes (applications) that directly transfer data back and forth between the two processes:



Some advantages of using named pipes include:

- The Fortran portion is now a standalone application, that can do things that a static library cannot, like receiving and transmitting messages and notifications between the VB and Fortran programs (more on that below).
- There is no need to use the ISO_C_BINDING module for conversion of data; however, note that the Fortran program will still have to do parsing and conversion of the received data (more on that below).
- Another interesting advantage is that communications via a named pipe are not limited to two applications running on the same PC; it potentially could be configured to have one program on a local PC, and the other program on another PC via a shared network.
- Refer to the document “Fortran Interoperability: ISO_C_BINDING vs. Windows Named Pipes + JSON-RPC 2.0” for a more in-depth discussion of this topic.

Data transfer

OK, so we have a way to transfer data between VB and Fortran programs. But, as the name ‘named pipe’ implies, it is just a ‘pipe’; it can transfer information, but not provide for the control of the flow of information. We need something to allow the VB and Fortran programs to ‘talk’ to each other without problems like simultaneous transmissions, letting the other program know what is being sent, when the transmission is complete, etc. For this application, the use of the JSON RPC 2.0 protocol standard was chosen. This provides a ‘lightweight’ network communication protocol (although the definition of ‘lightweight’ is subject to interpretation) that works with named pipes to automate the bidirectional communications between the two programs. It uses simple text based formats to transmit information and other functions between two processes.

But exactly what is the ‘JSON RPC 2.0 protocol standard’?

JSON: JSON stands for **JavaScript Object Notation**, a **data format** used to represent structured information as key–value pairs. Programs use it to **exchange data** across languages and systems because it’s easy for humans to read and easy for machines to parse. It’s especially common in **web APIs**, configuration files, and inter-process communication.

JSON-RPC 2.0 is a **remote procedure call protocol** that uses JSON messages to let one program invoke functions in another program as if they were local. It defines a simple structure for **requests and responses**, including method names, parameters, and return values.

As noted previously the JSON RPC protocol automatically acts as a traffic cop for the networks communications between the two programs. But the JSON RPC 2.0 standard (freely available on the Internet for download), delineates the formatting of the ‘messages’ sent between the programs. As an example of the relative simplicity of the format, below is an example message that sends the text ‘this is a message’ from one process to another:

```
{
  "jsonrpc": "2.0",
  "method": "sendMessage",
  "params": { "text": "this is a message" },
  "id": 1
}
```

It is left to the user to do further study of the JSON format and JSON RPC standard, but it is relatively easy to understand and implement, and hence the decision to use on this project.

Issues

Therefore, to implement this communication alternative for VB and Fortran, one just has to add the use of named pipes and JSON RPC 2.0 protocol, and all problems are solved, right? Alas, as with many things in real life, some issues have to be addressed; these include (but are not limited to):

- JSON RPC library for Fortran: Currently about 20 or so languages have support for implementation of the JSON RPC 2.0 standard in their programs. Unfortunately, Fortran is not on the list. This project developed a proof of concept JSON RPC 2.0 compatible static library to address this issue. Note that this static library is not 'complete' or 'optimized', but it works, and the source code is made available for others for further development and refinement.
- All text sent by VB via JSON RPC to the Fortran program is stored internally as **UTF-16 encoded Unicode**, meaning each character is represented as 16-bit code units that can express essentially all written languages. Unfortunately, Fortran is not one of these languages (perhaps the next standard revision will address this). So when receiving characters or strings from VB, the Fortran program must parse and convert the UTF formatted text into the older ASCII format.
- As with any computer language, the programs have different definitions of their defined variable types. Refer to the document "Marshalling of data between Fortran and Visual Basic: for a more in-depth discussion of the exchange of data between languages.
- There are other issues and compromises in this proof concept project, and the list of compliance with the JSON RPC 2.0 standard is provided in the document "JSON RPC 2.0 Standard Compliance Assessment".

Project programs

There are four programs associated with this project:

1. VB client: a VB.NET program that serves as the JSON RPC 'client' program. This is a 'demo' of various features and functions that demonstrate how the JSON RPC standard can be used for inter-process communications with a Fortran server.
2. VB JSON RPC DLL: a dynamic link library that was developed to assist in the development of JSON RPC compliant programs with commonly used functions. Note that this does NOT replace the StreamJsonRpc library.
3. Fortran server: a Fortran program (not a library) that provides the server function for JSON RPC based inter-process communications with the VB.NET client (or any other language that uses JSON RPC). It contains various 'methods' that the client program will call for, and provide the appropriate response. The demo version can also independently send 'notifications' to the client program while running, and allow the client program to 'pause/resume' or even 'exit' a method that is executing.
4. Fortran static library: This is a 'proof of concept' static library, written in Fortran to allow for the development of JSON RPC 2.0 standard compliant (mostly) programs.

5. As noted previously, refer to the document “JSON RPC 2.0 Standard Compliance Assessment” for more details how ‘compliant’ these programs are with the standard.
6. Refer to the document “Master List Of Filenames, Modules, Submodules, Functions, Subroutines, External References and Methods” for in-depth details on the programs and their internal features.
7. Last but not least, refer to “FORTRAN JSON RPC 2.0 Static Library Demonstration Programs User Manual” for an in-depth discussion of how the executables interact for the demonstration programs.
8. All program development was done on Visual Studio 2022 Community development platform. The most current version of Visual Basic provided by Microsoft was used (although it is acknowledged that Microsoft stopped active development of VB.NET many years ago). The most current version (first quarter 2026) of the Intel Fortran Compiler (IFX) was used with Visual Studio to develop the Fortran server and static library files.
9. AI used in the development of this project include GitHub Copilot (within Visual Studio), and Claude Pro (external to Visual Studio), fully acknowledging that this project could not be completed in six months without either tool.